

KNOWLEDGE-BASED AGENT

From Harness to Knowledge

실행할 수 있는 Agent에서, 경험을 축적하고 다시 쓰는 Agent로

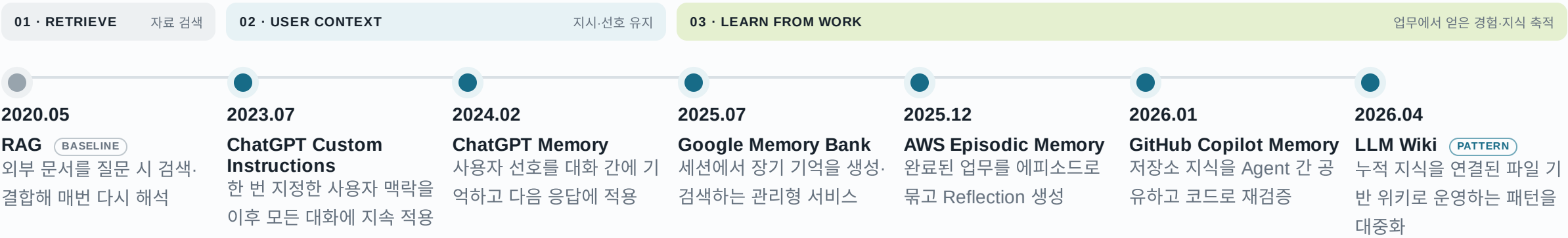
TODAY'S ROUTE

오늘 이야기의 흐름

01	Agent Memory의 변화	검색에서 지식 축적으로
02	우리가 확인한 것	Harness의 효과와 남은 손실
03	무엇을 남길 것인가	Knowledge의 종류와 재사용 조건
04	어떻게 일반화할 것인가	업무 흐름에 연결된 Knowledge Loop

SIGNALS IN PRACTICE

Agent가 기억하는 단위는 사용자 맥락에서 업무 경험과 지식으로 넓어지고 있다



제품과 패턴은 서로 대체되지 않으며, 저장 형식과 적용 범위가 다르다.

WHAT SURVIVES AFTER THE TASK?

차이는 업무가 끝난 뒤 무엇이 다음 실행에 남는가다



저장 형식은 구현에 따라 다르다 Memory Record · 저장소 사실 · 에피소드 · 위키 페이지

변화의 본질은 Agent가 업무 중 얻은 새로운 지식이 다음 실행에 남는다는 것이다.

THE BRIDGE

이름은 최근에 등장했지만, 우리는 이미 비슷한 문제를 다뤘다

외부의 새 패턴

경험을 구조화하고
다음 실행에 재사용



우리의 Harness 경험

반복 업무를 도구·검증·
절차로 고정하고 재사용

우리는 먼저 Agent가 일할 수 있는 실행 환경을 만들었다.

CASE 01 · INVESTIGATION

반복 조사: 매번 찾던 경로를 실행 가능한 절차로 바꿨다



남기지 못한 **Knowledge** 어떤 신호를 우선 보았는지, 잘못된 단서는 무엇이었는지, 예외 상황에서 경로를 왜 바꿨는지

변경 검증: 판단과 확인을 Guardrail 안에 넣었다



남기지 못한 **Knowledge** 왜 이 검사가 필요한지, 어떤 실패는 허용되는지, 변경 유형별 최소 검증 범위는 무엇인지

CASE 03 · REUSE

반복 실행: 한 번의 성공 경험을 다음 업무의 기본값으로 만들었다



남기지 못한 **Knowledge** 재사용 가능한 범위, 환경별 차이, 실패 후 얻은 교훈, 더 이상 유효하지 않은 조건

WHAT TO ACCUMULATE

우리가 쌓아야 하는 것은 결과물이 아니라 다음 판단과 행동을 바꾸는 지식이다

01

무엇이 사실인가

시스템 사실과 구조

구성 · 의존성 · 상태 · 제약

02

왜 이렇게 판단했는가

결정과 근거

선택 · 대안 · 트레이드오프

03

어떻게 실행하고 확인하는가

절차와 검증 방법

순서 · 도구 · 성공 기준

04

언제 성공하고 실패하는가

경험과 예외

적용 조건 · 실패 패턴 · 교훈

코드 · 테스트 · 설정 · 문서 · 로그 · Git 이력 Knowledge의 종류가 아니라, 위 지식을 발견하고 검증하는 원천이다

REUSABLE KNOWLEDGE

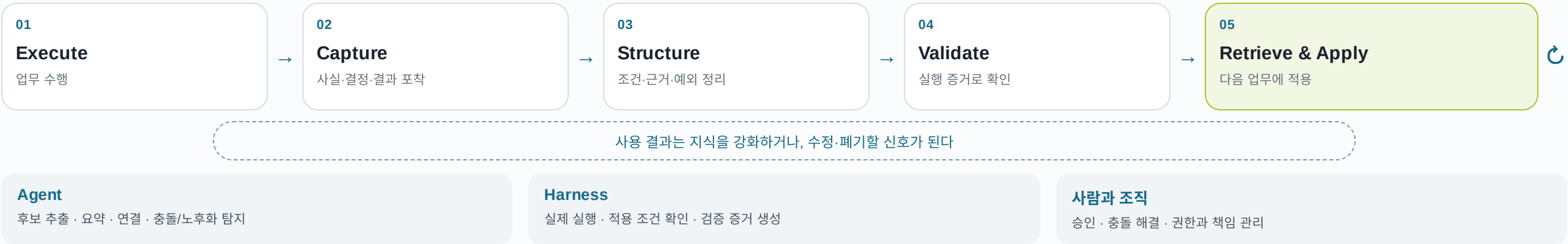
저장됐다고 모두 재사용 가능한 Knowledge가 되는 것은 아니다

SOURCE 어디서 왔는가 근거와 원천에 연결	SCOPE 어디에 적용되는가 대상과 조건을 명시	RATIONALE 왜 유효한가 판단 근거와 대안 기록
VALIDATION 확인됐는가 실행 결과와 검증 증거	FRESHNESS 아직 유효한가 시점과 노후화 신호 관리	OWNERSHIP 누가 책임지는가 검토 · 수정 · 폐기 권한

Reusable Knowledge = Content + Provenance + Applicability + Lifecycle

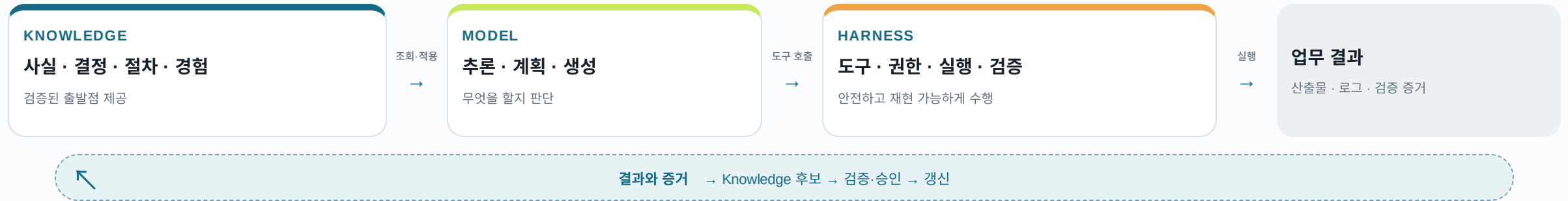
KNOWLEDGE OPERATING LOOP

일반화하려면 Knowledge 수집을 별도 문서화가 아니라 업무 흐름에 넣어야 한다



GENERALIZED SYSTEM

Model · Harness · Knowledge가 하나의 학습 루프로 연결돼야 한다



일반화의 핵심은 새 문서 저장소가 아니라, 축적된 지식을 실행과 피드백에 연결하는 것이다.

FROM CASES TO SYSTEM

개별 사례에서 확인한 Knowledge 재사용을 조직적으로 반복 가능한 구조로 만든다

WHAT

무엇을 남길지 정의한다

사실 · 결정 · 절차 · 경험

QUALITY

쓸 수 있는 조건을 붙인다

출처 · 범위 · 검증 · 생명주기

SYSTEM

업무 흐름에 연결한다

수집 · 재사용 · 검토를 Harness 안에서 반복

Execute → Observe → Distill → Validate → Reuse

다음 단계는 새로운 문서 저장소를 만드는 것이 아니라, 업무에서 얻은 판단과 경험이 다음 실행을 계속 개선하도록 연결하는 것이다.

용어를 이렇게 구분한다

용어	이 발표에서의 의미	경계
Memory	Agent가 이후 상호작용에 활용하도록 유지하는 상태	모든 기억이 조직 Knowledge는 아님
RAG	외부 자료에서 관련 맥락을 검색해 생성에 제공	검색만으로 축적·갱신은 보장되지 않음
Wiki Memory	업무 중 편집·연결·갱신되는 지속적 지식 구조	아직 단일한 업계 표준은 아님
Knowledge	출처·범위·검증을 갖고 다음 판단을 바꾸는 경험 자산	LLM 출력 자체와 구분

왜 지금인가: 비용 구조가 달라졌다

Capture

업무의 로그·결정·실패에서 후보를 상시 추출

Distill

비정형 경험에서 조건·근거·예외를 구조화

Maintain

사용 시점에 충돌과 노후화 신호를 다시 생성

사람의 승인 비용은 남는다. 달라진 것은 그 앞단의 정리 비용이다.

초기 연구는 구현 가능성을 보여주는 참고자료로만 본다

Retrieval as Reasoning · Tencent WeChat 연구팀 · arXiv preprint (2026.05)

Agent가 검색·읽기·링크 탐색을 반복하고, 오류 패턴을 다음 지식 구축에 반영하는 LLM-Wiki 시스템을 제시한다.

해석 주의 공개 직후의 자체 평가이며 확립된 업계 표준이 아니다. 벤치마크 수치를 우리 업무 효과로 전이하지 않고, 내부 효과는 별도 기준선과 반복 측정으로 검증한다.

가장 큰 위험은 더 많은 문서가 아니라 잘못된 확신이다

Hallucination

후보마다 원천 링크와 인용 범위 유지

Staleness

작성·검증 시점과 만료 신호 관리

Scope leak

프로젝트·환경·권한 경계를 메타데이터로 고정

Feedback loop

자기 생성 문장을 원천 사실처럼 재학습하지 않음

자동화의 대상은 지식의 확정이 아니라 후보 생성과 유지 신호다.

작게 시작하는 90일 검증

0-30 days

한 업무 선택

반복 빈도가 높고 검증 가능한 Harness 흐름 하나를 기준선으로 설정

31-60 days

Knowledge Contract 적용

출처·범위·시점·행동·책임 필드를 갖춘 후보 생성과 승인 실험

61-90 days

재사용 효과 비교

탐색 단계, 사람 개입, 재조사, 오류·철회 건수를 기준선과 비교

판단 기준 · 속도만이 아니라 정확성, 철회 가능성, 근거 추적성, 유지 비용을 함께 본다.

References

Enterprise Agent memory

[Google Cloud. “Vertex AI Memory Bank.” Jul 2025.](#)

[AWS. “Bedrock AgentCore GA.” Oct 2025.](#)

[GitHub. “Agentic memory for Copilot.” Jan 2026.](#)

[Anthropic. “Memory tool.” Claude Platform Docs.](#)

Knowledge management & governance

[NASA / University of Maryland. “The Experience Factory.” 1994/1995.](#)

[Consortium for Service Innovation. “KCS v6 Practices Guide.”](#)

[ISO/DIS 30401, 2nd ed. Knowledge management systems.](#)

[NIST AI 600-1. Generative AI Profile. 2024.](#)